

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:		Reg. No.:	
Date:		Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

- 1. Study and Analyze the Scenario**
 - Carefully understand the given problem statement.
 - Identify the business domain, stakeholders, existing challenges, and system objectives.
- 2. Identify Functional and Non-Functional Requirements**
 - List all functional requirements required by the proposed system.
 - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
- 3. Identify Actors and Use Cases**
 - Identify all primary and secondary actors interacting with the system.
 - List the major use cases associated with each actor.
 - Draw a Use Case Diagram representing the interactions between actors and the system.
- 4. Prepare the Software Requirement Specification (SRS)**

Develop an SRS document containing:

 - Introduction
 - Problem Description

- Scope of the System
 - Functional Requirements
 - Non-Functional Requirements
 - Use Case Descriptions
 - Data Requirements
 - External Interface Requirements
 - System Features
5. **Identify Assumptions and Constraints**
- List the assumptions considered while developing the system.
 - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
- Verify whether all stakeholder requirements have been addressed.
 - Check for ambiguity, redundancy, inconsistency, and missing requirements.
 - Suggest improvements to enhance the quality and completeness of the requirement specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modeling	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification	SRS is well-structured, complete, professionally	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or

(SRS) Documentation	organized, and includes all required sections.			lacks essential components.
Assumptions, Constraints, and Requirement Validation	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	Mark
Scenario Understanding and Problem Analysis	/5
Functional and Non-Functional Requirement Identification	/5
Actor Identification and Use Case Modeling	/5
Software Requirement Specification (SRS) Documentation	/5
Assumptions, Constraints, and Requirement Validation	/5
TOTAL	/5

PROBLEM STATEMENT : AI-Powered Digital Twin Platform for Smart City Infrastructure

Industry Context

Smart cities increasingly adopt digital twin technology to simulate and monitor urban infrastructure in real time. AI, IoT, GIS, and cloud computing enable virtual replicas of roads, utilities, buildings, and transportation systems for predictive maintenance and strategic planning.

Problem Statement

Urban infrastructure management is challenging due to aging assets, increasing population, and limited real-time visibility. Traditional monitoring systems provide fragmented information and lack predictive capabilities. An AI-powered digital twin platform should create virtual models of city infrastructure by integrating real-time IoT data. The system should simulate infrastructure performance, detect anomalies, predict failures, and support data-driven urban planning. This solution will improve infrastructure reliability and maintenance efficiency.

Objectives

1. Create digital twins of city infrastructure.
2. Integrate real-time IoT data.
3. Predict infrastructure failures.
4. Monitor asset health continuously.
5. Support predictive maintenance.
6. Simulate urban scenarios.
7. Generate infrastructure performance reports.
8. Improve smart city planning.

Expected Outcomes

1. Enhanced infrastructure monitoring.
2. Reduced maintenance costs.
3. Improved asset reliability.
4. Better urban planning decisions.
5. Accurate predictive maintenance.
6. Increased operational efficiency.
7. Real-time infrastructure visualization.
8. Sustainable city development.

1. Study and Analyze the Scenario

1.1 Introduction

Smart cities require efficient monitoring and management of infrastructure such as roads, bridges, buildings, utilities, and transportation systems. Traditional monitoring methods often depend on manual inspection and fragmented data, making it difficult to identify problems before failures occur.

The proposed AI-Powered Digital Twin Platform creates virtual representations of physical city infrastructure and continuously updates them using real-time IoT sensor data.

1.2 Business Domain

The system belongs to the Smart City and Urban Infrastructure Management domain and integrates: **AI/ML, Digital Twin Technology, IoT, GIS and Cloud Computing.**

1.3 Existing Challenges

- Lack of real-time infrastructure monitoring.
- Infrastructure information is stored in fragmented systems.
- Maintenance is mostly reactive rather than predictive.
- Manual inspections require significant time and resources.
- Existing systems have limited failure-prediction capabilities.

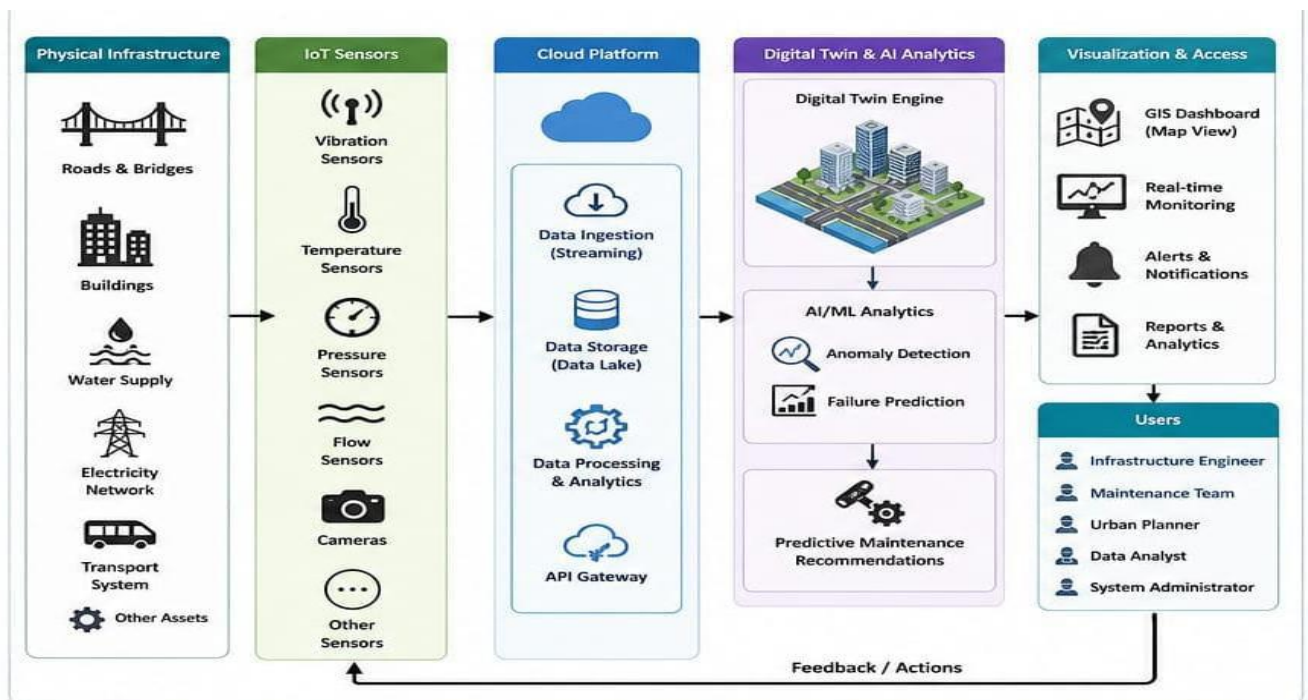


FIG 1: ARCHITECTURE DIAGRAM

1.4 Stakeholders

Stakeholder	Responsibility
Smart City Authority	Supervises infrastructure and makes decisions
System Administrator	Manages users, assets and system configuration
Infrastructure Engineer	Monitors infrastructure health
Maintenance Team	Performs maintenance based on alerts
Urban Planner	Performs simulations and planning
Data Analyst	Analyzes historical and performance data
IoT Sensor Network	Supplies real-time infrastructure data

1.5 System Objectives

The system aims to:

1. Create digital twins of city infrastructure.
2. Integrate real-time IoT data.
3. Monitor infrastructure health continuously.
4. Detect anomalies and predict failures using AI.
5. Support predictive maintenance.
6. Simulate urban scenarios.
7. Generate infrastructure performance reports.
8. Improve smart-city planning.

2. Functional and Non-Functional Requirements

The assessment specifically requires both functional and non-functional requirements covering areas such as performance, security, reliability, usability, scalability, maintainability and availability.

2.1 Functional Requirements

ID	Functional Requirement
FR1	The system shall authenticate authorized users.
FR2	The system shall register and manage infrastructure assets.
FR3	The system shall create digital twins of physical infrastructure.
FR4	The system shall collect real-time data from IoT sensors.
FR5	The system shall continuously monitor asset health.
FR6	The system shall detect anomalies using AI.
FR7	The system shall predict possible infrastructure failures.
FR8	The system shall generate maintenance recommendations and alerts.
FR9	The system shall display infrastructure through GIS maps.
FR10	The system shall support urban scenario simulation.

2.2 Non-Functional Requirements

Category	Requirement
Performance	Sensor data should be processed with minimum delay.
Security	Authentication, authorization and encryption shall protect infrastructure data.
Reliability	The system should provide accurate and consistent monitoring information.
Availability	Critical monitoring services should remain continuously available.
Scalability	The system should support increasing numbers of sensors and assets.
Usability	Dashboards and GIS interfaces should be easy to understand.
Maintainability	Components should be modular and easy to update.
Interoperability	The system should integrate with different IoT and GIS services.

3. Actors and Use Cases

3.1 Actors

Primary Actors

System Administrator

- Manage users and IoT devices
- Register infrastructure

Infrastructure Engineer

- Monitor infrastructure
- View digital twins and asset health
- Analyze failure predictions
- Receive alerts

Maintenance Team

- View maintenance alerts
- Perform maintenance
- Update maintenance status

Urban Planner

- View GIS map and reports
- Run urban simulations

Data Analyst

- Analyze historical data
- Generate reports

Secondary Actors

- IoT Sensor Network
- GIS Service
- Cloud Database

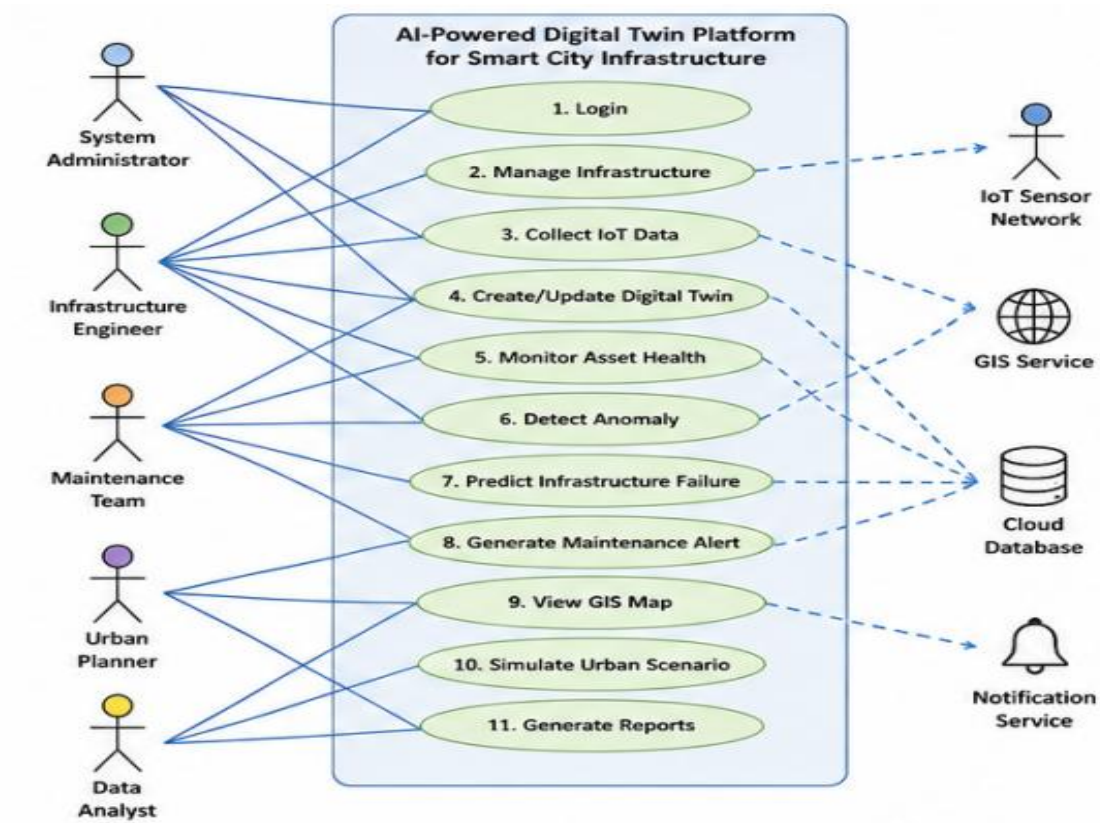


FIG 2 : USE CASE DIAGRAM

3.2 Major Use Cases

1. Login
2. Manage infrastructure
3. Collect IoT data
4. Create/update digital twin
5. Monitor asset health
6. Detect anomaly
7. Predict infrastructure failure
8. Generate maintenance alert
9. View GIS map
10. Simulate urban scenario
11. Generate reports

4. Software Requirement Specification (SRS)

4.1 Problem Description

Urban infrastructure management is difficult because of aging assets, population growth and limited real-time monitoring. Traditional systems provide fragmented information and generally lack predictive capabilities. The proposed system integrates Digital Twins, IoT and AI to continuously monitor infrastructure, identify abnormalities and predict possible failures before they become critical.

4.2 Scope of the System

The system covers:

- Infrastructure asset management
- Digital twin creation
- Real-time IoT monitoring
- AI-based anomaly detection
- Failure prediction
- Predictive maintenance
- GIS visualization
- Urban simulation
- Performance reporting

4.3 Use Case Description

UC1 – Monitor Infrastructure Health

Field	Description
Actor	Infrastructure Engineer
Precondition	User is logged in and sensors are active
Main Flow	Collect sensor data → Update digital twin → Analyze asset condition → Display health status
Postcondition	Current infrastructure health is displayed

UC2 – Predict Infrastructure Failure

Field	Description
Actor	Infrastructure Engineer
Precondition	Current and historical data are available
Main Flow	AI analyzes data → Detects patterns → Calculates failure risk → Displays prediction
Postcondition	Failure prediction is stored and displayed

UC3 – Urban Scenario Simulation

Field	Description
Actor	Urban Planner
Precondition	Digital twin data is available
Main Flow	Select infrastructure → Enter scenario → Run simulation → Display results
Postcondition	Results are available for planning

5. Data and External Interface Requirements

These sections are explicitly included in the SRS requirements given in your assessment.

5.1 Data Requirements

The system stores:

Asset Data: Asset ID, type, location, status and maintenance history.

Sensor Data: Sensor ID, readings, timestamp, temperature, vibration, pressure and other parameters.

Digital Twin Data: Twin ID, corresponding asset and current virtual state.

Prediction Data: Failure probability, severity and recommended action.

User Data: User ID, role and access permissions.

Maintenance Data: Maintenance activity, date and status.

5.2 External Interface Requirements

User Interface: Web dashboard for monitoring infrastructure.

IoT Interface: Receives real-time information from sensors.

GIS Interface: Provides geographical visualization.

AI Interface: Processes data for anomaly detection and failure prediction.

Database Interface: Stores infrastructure, sensor and historical information.

6. Assumptions and Constraints

6.1 Assumptions

- IoT sensors are installed on infrastructure assets.
- Sensors provide accurate and timely data.
- Reliable network connectivity is available.
- Historical data is available for AI analysis.
- GIS information is available.
- Authorized users have valid login credentials.
- Cloud infrastructure provides sufficient storage and processing capacity.

6.2 Constraints

Technical: Dependence on IoT devices, network connectivity and cloud services.

Economic: Installation and maintenance of sensors may be expensive.

Operational: Sensors require proper installation and calibration.

Data: AI prediction accuracy depends on data quality and availability.

Security: Sensitive infrastructure information must be protected.

Legal: Infrastructure data must follow applicable privacy and government regulations.

7. Requirement Validation

Completeness

All major objectives are covered by functional requirements, including digital twins, IoT integration, failure prediction, monitoring, maintenance, simulation and reporting.

Consistency

Requirements have been checked to ensure that they do not contradict each other. Each actor has clearly defined responsibilities.

Ambiguity

Requirements should use clear and measurable terms. Terms such as *failure*, *critical condition* and *real-time* should have defined thresholds during implementation.

Redundancy

Similar requirements are separated appropriately. For example, **anomaly detection** identifies abnormal behaviour, while **failure prediction** estimates future failure risk.

7.1 Requirement Traceability

Objective	Requirement
Create Digital Twins	FR3
Integrate IoT Data	FR4
Monitor Asset Health	FR5
Detect Anomalies	FR6
Predict Failures	FR7
Predictive Maintenance	FR8
Urban Simulation	FR10
Performance Reports	FR11

8. Conclusion

The **AI-Powered Digital Twin Platform for Smart City Infrastructure** combines AI, IoT, GIS and digital twin technology to provide intelligent infrastructure management. The system enables real-time monitoring, anomaly detection, failure prediction, predictive maintenance and urban simulation.

By replacing fragmented and reactive monitoring with a centralized predictive approach, the proposed platform can improve infrastructure reliability, reduce maintenance costs, increase operational efficiency and support sustainable smart-city planning.